

Neural Tangent Kernel: Convergence, Generalisation, Adaptive Optimisers, and Post-2018 Developments

Based on Jacot, Gabriel & Hongler, NeurIPS 2018

Summary & Extensions

May 27, 2026

Outline

- 1 Motivation & Background
- 2 The Neural Tangent Kernel
- 3 Convergence & Generalisation
- 4 Connection to Natural Gradient
- 5 Connection to Stochastic Reconfiguration
- 6 Neural Tangent Gradients & Adaptive Optimisers
- 7 Post-2018 Developments
- 8 Summary & Outlook

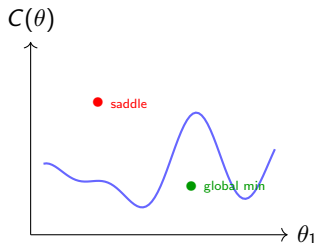
Why Study Neural Network Training Dynamics?

Open questions circa 2018:

- Loss landscape is **highly non-convex**: many saddle points, potential bad local minima.
- Despite over-parametrisation, ANNs **generalise remarkably well**.
- The dynamics of *deep* networks during gradient descent were not understood.

Known connections:

- At initialisation, infinite-width ANNs $\equiv$ **Gaussian Processes** (Neal 1996; Lee et al. 2018).
- Kernel methods achieve comparable performance (Cho & Saul, 2009).



Key Insight

Study training in **function space** $\mathcal{F}$, where the cost is **convex**, rather than in non-convex parameter space.

ANN Setup and Notation

Fully-connected ANN, L layers, widths $n_0, n_1, \dots, n_L$.

Preactivations and activations ($\ell = 0, \dots, L - 1$):

$$\alpha^{(0)}(x) = x, \quad \tilde{\alpha}^{(\ell+1)}(x) = \frac{1}{\sqrt{n_\ell}} W^{(\ell)} \alpha^{(\ell)}(x) + \beta b^{(\ell)}, \quad \alpha^{(\ell+1)} = \sigma\left(\tilde{\alpha}^{(\ell+1)}\right).$$

Parameters $W_{ij}^{(\ell)}, b_j^{(\ell)} \sim \mathcal{N}(0, 1)$ i.i.d. at initialisation.

Network function: $f_\theta(x) := \tilde{\alpha}^{(L)}(x; \theta) \in \mathbb{R}^{n_L}$

Function space $\mathcal{F} = \{f : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L}\}$ with seminorm

$$\langle f, g \rangle_{p^{\text{in}}} = \mathbb{E}_{x \sim p^{\text{in}}} [f(x)^\top g(x)].$$

Key normalisation

The factor $1/\sqrt{n_\ell}$ is crucial for a consistent infinite-width limit. It ensures that preactivations remain $\mathcal{O}(1)$ as width grows.

Kernel Gradient Descent

For a multi-dimensional kernel $K : \mathbb{R}^{n_0} \times \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L \times n_L}$, define the **kernel gradient**:

$$\nabla_K C|_{f_0}(x) = \frac{1}{N} \sum_{j=1}^N K(x, x_j) d|_{f_0}(x_j), \quad \partial_t f^{\text{in}} C|_{f_0} = \langle d|_{f_0}, \cdot \rangle_{p^{\text{in}}}.$$

Kernel gradient descent evolves $f(t)$ via:

$$\partial_t f(t) = -\nabla_K C|_{f(t)}.$$

Theorem (Cost decay)

$$\partial_t C|_{f(t)} = -\|d|_{f(t)}\|_K^2 \leq 0.$$

If K is **positive definite** w.r.t. $\|\cdot\|_{p^{\text{in}}}$ and C is convex and bounded below, then $f(t)$ converges to a **global minimum**.

The Neural Tangent Kernel (NTK)

During gradient descent on $C \circ F^{(L)}$, the network function evolves as:

$$\partial_t f_\theta(t) = -\nabla_{\Theta^{(L)}(\theta)} C|_{f_\theta(t)},$$

where the **Neural Tangent Kernel** is:

$$\Theta^{(L)}(\theta) = \sum_{p=1}^P \partial_{\theta_p} F^{(L)}(\theta) \otimes \partial_{\theta_p} F^{(L)}(\theta).$$

For finite networks:

- NTK is **random** at initialisation.
- NTK **varies** during training.
- Analysis is delicate.

In the infinite-width limit:

- NTK becomes **deterministic**.
- NTK remains **constant** during training.
- Dynamics fully tractable!

Infinite-Width Limit: Gaussian Processes

Proposition 1 (Infinite-width GP limit)

As $n_1, \dots, n_{L-1} \rightarrow \infty$, the output functions f_{θ_k} tend (in law) to i.i.d. centred Gaussian processes with covariance $\Sigma^{(L)}$, defined recursively:

$$\Sigma^{(1)}(x, x') = \frac{1}{n_0} x^\top x' + \beta^2,$$

$$\Sigma^{(L+1)}(x, x') = \mathbb{E}_{f \sim \mathcal{N}(0, \Sigma^{(L)})} [\sigma(f(x)) \sigma(f(x'))] + \beta^2.$$

This extends Neal (1996) to deep networks. The kernel $\Sigma^{(L)}$ is the **NNGP kernel**.

Interpretation

At initialisation, an infinitely wide ANN is *equivalent* to Bayesian inference with a GP prior — connecting deep learning to classical kernel methods.

Theorem 1: NTK Convergence at Initialisation

Theorem (Jacot, Gabriel & Hongler, 2018)

As $n_1, \dots, n_{L-1} \rightarrow \infty$, the NTK converges in probability to a deterministic limit:

$$\Theta^{(L)}(\theta) \xrightarrow{P} \Theta_{\infty}^{(L)} \otimes \text{Id}_{n_L}.$$

The scalar kernel $\Theta_{\infty}^{(L)}$ is defined recursively:

$$\Theta_{\infty}^{(1)}(x, x') = \Sigma^{(1)}(x, x'),$$

$$\Theta_{\infty}^{(L+1)}(x, x') = \Theta_{\infty}^{(L)}(x, x') \dot{\Sigma}^{(L+1)}(x, x') + \Sigma^{(L+1)}(x, x'),$$

where $\dot{\Sigma}^{(L+1)}(x, x') = \mathbb{E}_{f \sim \mathcal{N}(0, \Sigma^{(L)})} [\dot{\sigma}(f(x)) \dot{\sigma}(f(x'))]$.

The limiting NTK depends only on: depth L , nonlinearity σ , initialisation variance.

Theorem 2: NTK Stays Constant During Training

Theorem (Stability of NTK under training)

Under mild regularity assumptions on σ , and for any T such that $\int_0^T \|d_t\|_{p^{\text{in}}} dt$ stays stochastically bounded, as $n_1, \dots, n_{L-1} \rightarrow \infty$:

$$\Theta^{(L)}(t) \xrightarrow{P} \Theta_{\infty}^{(L)} \otimes \text{Id}_{n_L}, \quad \text{uniformly for } t \in [0, T].$$

Consequence: In the infinite-width limit, ANN gradient descent *is* kernel gradient descent with a **fixed kernel** $\Theta_{\infty}^{(L)}$:

$$\partial_t f_{\theta}(t) = \Phi_{\Theta_{\infty}^{(L)} \otimes \text{Id}}(\langle d_t, \cdot \rangle_{p^{\text{in}}}).$$

Mechanistic picture

Individual neuron activations change little, but their *collective* variation drives learning in lower layers.

In the NTK recursion $\Theta_{\infty}^{(L+1)} = \Theta_{\infty}^{(L)} \dot{\Sigma}^{(L+1)} + \Sigma^{(L+1)}$:
 $\Sigma^{(L+1)} \rightarrow$ learning in last layer,
 $\Theta_{\infty}^{(L)} \dot{\Sigma}^{(L+1)} \rightarrow$ learning in lower layers.

Least-Squares Regression: Exact Solution

For the cost $C(f) = \frac{1}{2} \|f - f^*\|_{p^{\text{in}}}^2$, the dynamics reduce to a **linear ODE**:

$$\partial_t f_t = \Phi_K(\langle f^* - f_t, \cdot \rangle_{p^{\text{in}}}) = -\Pi(f_t - f^*),$$

where $\Pi(f)_k(x) = \frac{1}{N} \sum_{i=1}^N \sum_{k'} f_{k'}(x_i) K_{kk'}(x_i, x)$.

Exact solution:

$$f_t = f^* + e^{-t\Pi}(f_0 - f^*).$$

Decomposing along eigenfunctions $f^{(i)}$ of Π with eigenvalues λ_i :

$$f_t = f^* + \Delta_f^0 + \sum_{i=1}^{Nn_L} e^{-\lambda_i t} \Delta_f^i.$$

Key observation

Convergence is *fastest* along directions with **largest eigenvalues** λ_i — the leading kernel principal components of the data.

Generalisation and Early Stopping

As $t \rightarrow \infty$, the solution converges to:

$$f_{\infty,k}(x) = \kappa_{x,k}^\top \tilde{K}^{-1} y^* + \underbrace{\left(f_0(x) - \kappa_{x,k}^\top \tilde{K}^{-1} y_0 \right)}_{\text{centred Gaussian, zero on data}},$$

where $\tilde{K} = (K_{kk'}(x_i, x_j))$ is the Gram matrix.

The **mean** $\kappa_{x,k}^\top \tilde{K}^{-1} y^*$ equals:

- **MAP estimate** with GP prior $f_k \sim \mathcal{N}(0, \Theta_\infty^{(L)})$.
- **Kernel ridge regression** (regularisation $\lambda \rightarrow 0$).

Motivation for Early Stopping

- Low- λ_i directions correspond to **high-frequency / noisy** components (e.g. high-frequency modes for RBF kernel).
- Early stopping halts training before fitting these directions.
- $\Rightarrow$ implicit regularisation towards **smooth, low-complexity** solutions.

Numerical Evidence

NTK convergence (unit circle, $L = 4$):

- Wider networks: smaller NTK variance, smoother kernel.
- Training causes NTK to “inflate” for finite-width; negligible for $n = 10\,000$.

Kernel regression distribution:

- Network output distribution at convergence closely matches theoretical GP, even for $n = 50$.

Principal component convergence (MNIST):

- Eigenvalues: $\lambda_1 \gg \lambda_2 \approx \lambda_3$.
- Wider networks converge more *linearly* (in function space) toward f^* .
- **Surprising:** narrower networks appear to converge faster in step count — explained by NTK inflation effectively increasing the learning rate.

Natural Gradient — Review

Ordinary gradient descent in parameter space:

$$\dot{\theta} = -\nabla_{\theta} C(\theta).$$

This is metric-dependent: changing parametrisation changes the path.

Natural gradient (Amari, 1998) uses the **Fisher information metric** $F(\theta)$:

$$\boxed{\dot{\theta} = -F(\theta)^{-1} \nabla_{\theta} C(\theta)}, \quad F_{pq}(\theta) = \mathbb{E}_x \left[\sum_k \partial_{\theta_p} \log p_{\theta}(k|x) \partial_{\theta_q} \log p_{\theta}(k|x) \right].$$

Key properties:

- Parametrisation-invariant — acts in the *distribution space*.
- Equivalent to steepest descent in the KL-divergence geometry.
- Encodes the curvature of the model manifold.

Cost

$F(\theta)^{-1}$ is $P \times P$ — prohibitive for large networks. Approximations (K-FAC, EKFAC, ...) are needed in practice.

NTK as a Fisher-like Metric

The NTK can be written as:

$$\Theta^{(L)}(\theta; x, x') = \sum_p \partial_{\theta_p} F^{(L)}(\theta)(x) \otimes \partial_{\theta_p} F^{(L)}(\theta)(x').$$

Compare with the Fisher matrix evaluated on data:

$$F_{pq}(\theta) = \frac{1}{N} \sum_{i,k} \partial_{\theta_p} f_{\theta,k}(x_i) \partial_{\theta_q} f_{\theta,k}(x_i).$$

The NTK is the “push-forward” of the Fisher metric from parameter space to function space:

$$\Theta^{(L)}(\theta) = J(\theta) J(\theta)^\top,$$

where $J(\theta) = \partial_\theta F^{(L)}(\theta)$ is the Jacobian.

Equivalence statement

Natural gradient descent in parameter space $\equiv$ kernel gradient descent in function space w.r.t. the NTK:

$$\dot{f}_\theta = -\Theta_\infty^{(L)} \partial_f C.$$

NTK vs Natural Gradient: Comparison

	Natural Gradient	NTK (infinite width)
Space	Parameter $\mathbb{R}^P$	Function $\mathcal{F}$
Metric	Fisher $F(\theta)$	$\Theta_{\infty}^{(L)}$ (fixed)
Evolution eq.	$\dot{\theta} = -F^{-1} \nabla_{\theta} C$	$\dot{f} = -\Theta_{\infty}^{(L)} \partial_f C$
Curvature	Changes with θ	Constant (infinite width)
Convergence	Depends on spectrum of F	Depends on spectrum of $\Theta_{\infty}^{(L)}$
Practical cost	$\mathcal{O}(P^2)$ inversion	$\mathcal{O}(N^2)$ kernel matrix

Insight: In the infinite-width limit, natural gradient descent *automatically* reduces to a fixed-kernel method, removing the need to track curvature changes.

Stochastic Reconfiguration (SR) — Quantum Context

In **variational Monte Carlo** (VMC), one optimises a parametrised wave function Ψ_θ to minimise the variational energy $E[\Psi_\theta] = \langle \Psi_\theta | H | \Psi_\theta \rangle / \langle \Psi_\theta | \Psi_\theta \rangle$.

Stochastic Reconfiguration (Sorella, 1998) updates parameters via:

$$\boxed{\dot{\theta} = -S(\theta)^{-1} g(\theta)}, \quad S_{pq}(\theta) = \mathbb{E}_{\Psi_\theta} [O_p^*(x) O_q(x)] - \mathbb{E}[O_p^*] \mathbb{E}[O_q],$$

where $O_p(x) = \partial_{\theta_p} \log \Psi_\theta(x)$ are the **logarithmic derivatives**, and $g_p = \partial_{\theta_p} E$.

$S(\theta)$ is the **quantum geometric tensor** (overlap matrix / quantum Fisher information):

$$S_{pq} = \text{Re}(\langle \partial_p \Psi | \partial_q \Psi \rangle - \langle \partial_p \Psi | \Psi \rangle \langle \Psi | \partial_q \Psi \rangle).$$

Geometric interpretation

SR is natural gradient descent on the manifold of quantum states, respecting the Fubini-Study metric on the projective Hilbert space.

SR, Fisher, and the NTK: Unified View

All three methods implement **preconditioned gradient descent** with a metric matrix M :

Method	Domain	Metric M	Update
Gradient descent	$\mathbb{R}^P$	$\mathbf{I}$	$\dot{\theta} = -\nabla_{\theta} C$
Natural gradient	prob. models	Fisher $F(\theta)$	$\dot{\theta} = -F^{-1} \nabla C$
Stoch. Reconfiguration	quantum states	$S(\theta)$ (QFI)	$\dot{\theta} = -S^{-1} g$
NTK gradient (inf. width)	function space	$\Theta_{\infty}^{(L)}$ (fixed)	$\dot{f} = -\Theta_{\infty}^{(L)} \partial_f C$

Unifying observation

In all cases the metric encodes the *geometry of the model manifold*. The NTK is special because it **freezes** in the infinite-width limit, converting a nonlinear optimisation problem into a *linear* one — a property not generally shared by SR or the natural gradient.

SR with Neural Quantum States and NTK

Modern VMC uses **neural quantum states** (NQS, Carleo & Troyer 2017) — parametrised ANNs for Ψ_θ .

The SR matrix for an NQS becomes:

$$S_{pq} = \mathbb{E}_{\pi_\theta} [\partial_{\theta_p} \log |\Psi_\theta| \partial_{\theta_q} \log |\Psi_\theta|]_c \approx \frac{1}{N_s} \sum_{s=1}^{N_s} \bar{O}_p(x_s) \bar{O}_q(x_s), \quad \bar{O}_p = O_p - \langle O_p \rangle.$$

Connection to NTK:

- Define $J_{s,p} = \partial_{\theta_p} \log |\Psi_\theta(x_s)|$. Then

$$S \approx \frac{1}{N_s} J^\top J \quad (= \text{empirical NTK!})$$

- In the **infinite-width NQS limit**, S converges to a fixed deterministic kernel $\Theta_\infty^{(L)}$ — the NTK of the network.
- SR for wide NQS $\approx$ kernel gradient descent in function space.

Practical implications

- Spectrum of S (eigenvalue distribution) directly controls SR convergence speed — mirroring the NTK eigenvalue story.
- Ill-conditioning of S at near-zero eigenvalues corresponds to *redundant parameters* — diagnosed by the NTK.
- Regularisation ($S + \epsilon I$) corresponds to implicit early stopping.

Spectrum and Convergence: SR vs NTK

The NTK analysis gives a sharp picture of convergence rates:

$$\|f_t - f^*\|^2 \sim \sum_i e^{-2\lambda_i t} |\Delta_f^i|^2.$$

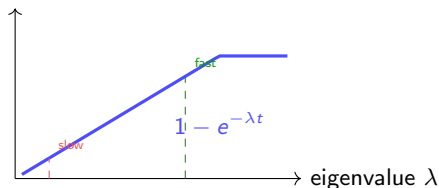
For NTK gradient descent:

- Eigenvalues λ_i of Π (kernel PCA eigenvalues) set convergence timescales.
- Large $\lambda_i \rightarrow$ fast convergence in that direction.
- Small $\lambda_i \rightarrow$ slow convergence (high-frequency modes).

For SR (NQS):

- Eigenvalues of S set update magnitudes.
- Near-zero eigenvalues: over-complete parametrisation.
- Shift $S + \epsilon I$ regularises (MINRES-SR, diagonal shift).

convergence rate



From NTK to Neural Tangent Gradients

The NTK describes how the *network function* evolves. But practitioners update *parameters* θ with gradient-based rules. This motivates asking:

What is the parameter-space gradient induced by the NTK?

Define the **Neural Tangent Gradient** (NTG) as the exact gradient of the cost projected through the tangent map of the network:

$$\mathbf{g}_p^{\text{NT}}(t) = \partial_{\theta_p} C(\theta(t)) = \sum_{i,k} \partial_{\theta_p} f_{\theta,k}(x_i) \delta_k(x_i),$$

where $\delta_k(x_i) = \partial_{f_k(x_i)} C$ is the output residual.

Vector form over the dataset:

$$\mathbf{g}^{\text{NT}} = J(\theta)^\top \boldsymbol{\delta}, \quad J_{(i,k),p} = \partial_{\theta_p} f_{\theta,k}(x_i).$$

Here $J \in \mathbb{R}^{N_{n_L} \times P}$ is the **full Jacobian** and $\boldsymbol{\delta} \in \mathbb{R}^{N_{n_L}}$ stacks the residuals.

Key relationship

The NTK is the *Gram matrix* of NTGs:

$$\Theta_{(i,k),(j,k')}^{(L)} = \sum_p \mathbf{g}_{p,(i,k)}^{\text{NT}} \mathbf{g}_{p,(j,k')}^{\text{NT}}.$$

NTK = $JJ^\top$ in function space; NTG = $J^\top \boldsymbol{\delta}$ in parameter space

Gradient Descent Through the NTK Lens

Ordinary gradient descent in parameter space reads:

$$\theta_{t+1} = \theta_t - \eta J(\theta_t)^\top \delta_t.$$

The induced update in *function space* is:

$$f_{\theta_{t+1}}(x) \approx f_{\theta_t}(x) - \eta \underbrace{J(x, \theta_t) J(\theta_t)^\top}_{\Theta^{(L)}(\theta_t; x, \cdot)} \delta_t,$$

where $J(x, \theta) = \partial_\theta f_\theta(x) \in \mathbb{R}^{n_L \times P}$.

Implications:

- The **effective step size** in function space is *not* η but $\eta \lambda_{\min}(\Theta)$ to $\eta \lambda_{\max}(\Theta)$, set by the NTK spectrum.
- Poor conditioning ($\lambda_{\max} \gg \lambda_{\min}$) causes oscillations in large-eigenvalue directions and stagnation in small ones.
- Adaptive optimisers address exactly this mismatch — they **implicitly rescale the NTG**

Why a fixed learning rate is suboptimal

A single η must simultaneously avoid divergence along the largest NTK eigenvalue *and* make progress along the smallest. The ratio $\kappa = \lambda_{\max}/\lambda_{\min}$ (condition number of Θ) controls the difficulty — wide networks with very non-uniform spectra suffer most.

Adam: Gradient-Only Adaptive Learning Rates

Adam (Kingma & Ba, 2015) maintains running estimates of the first and second moments of the gradient vector, using only gradient information:

Update rules (at step t , for each parameter p):

$$m_p^{(t)} = \beta_1 m_p^{(t-1)} + (1 - \beta_1) g_p^{(t)},$$

$$v_p^{(t)} = \beta_2 v_p^{(t-1)} + (1 - \beta_2) (g_p^{(t)})^2,$$

$$\hat{m}_p = \frac{m_p^{(t)}}{1 - \beta_1^t}, \quad \hat{v}_p = \frac{v_p^{(t)}}{1 - \beta_2^t},$$

$$\theta_p^{(t+1)} = \theta_p^{(t)} - \frac{\eta \hat{m}_p}{\sqrt{\hat{v}_p} + \epsilon}.$$

What each term does:

- m_p — exponential moving average of gradients (momentum / bias correction).
- v_p — exponential moving average of *squared* gradients (tracks per-parameter curvature).
- $1/\sqrt{\hat{v}_p}$ — **normalises** the update: large curvature $\Rightarrow$ smaller step.

Typical values

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}.$$

Adam as an Implicit NTK Preconditioner

Key observation: the diagonal of $J^\top J$ (i.e. the per-parameter squared NTG norms) governs how much each parameter contributes to the function-space update.

The **diagonal Fisher** approximation gives:

$$F_{pp} \approx \mathbb{E}_x \left[\left(\partial_{\theta_p} f_\theta(x) \right)^2 \right] = [J^\top J]_{pp}.$$

Adam's $v_p^{(t)} \approx \mathbb{E}[(g_p^{(t)})^2]$ **estimates exactly this diagonal**, so:

$$\frac{\eta}{\sqrt{v_p}} \approx \frac{\eta}{\sqrt{F_{pp}}} \quad \Leftrightarrow \quad \text{diagonal natural gradient step.}$$

In NTK language:

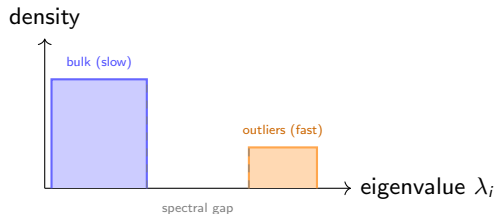
- Adam rescales the NTG *per-parameter*, adaptively matching the local curvature of the NTK without ever forming the full $P \times P$ matrix.
- Parameters θ_p that drive large function-space changes (large $[J^\top J]_{pp}$) receive **smaller effective learning rates**.
- Parameters in low-curvature directions receive

Why Adam works well in practice

The NTK spectrum of deep networks is typically very non-uniform. Adam's per-coordinate scaling partially corrects for this, acting as a cheap diagonal approximation to the full NTK-preconditioned update — at cost $\mathcal{O}(P)$ rather than $\mathcal{O}(P^2)$.

NTG, Adam, and the NTK Spectrum: A Unified View

Schematic: NTK eigenvalue spectrum



Bulk: many parameters with small NTG norm $\rightarrow$ slow convergence with SGD.

Outliers: few parameters with large NTG norm $\rightarrow$ fast convergence but instability risk.

Optimiser	Relation to NTK
SGD	Step $\propto \Theta_{\infty}^{(L)} \delta$ (uniform η , no rescaling)
Momentum	Smooths NTK estimates over time
RMSProp	Diagonal v_p scaling; estimates $[J^{\top} J]_{pp}$
Adam	Momentum + diagonal $[J^{\top} J]_{pp}$ scaling; bias-corrected
Nat. Grad. / SR	Full $[J^{\top} J]^{-1}$ in parameter space (exact NTK precondition.)
NTK GD	Full $\Theta_{\infty}^{(L)}$ in <i>function</i> space (exact, infinite)

Adam vs NTK: Limitations and Open Questions

What Adam captures:

- Diagonal curvature (second moment of NTG).
- Temporal smoothing via exponential moving averages.
- Bias correction for early-training transients.
- Implicit regularisation through bounded effective step sizes.

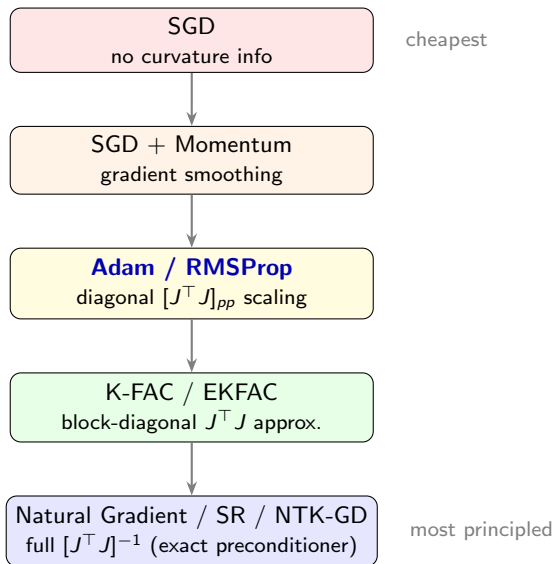
What Adam misses:

- **Off-diagonal NTK structure:** correlations between parameters are ignored.
- **Feature learning:** Adam (like NTK theory) is most natural in the lazy/linear regime; feature learning breaks the fixed-kernel picture.
- **NTK drift:** finite-width networks have a slowly drifting kernel; Adam's v_p tracks this implicitly but inexactly.

Practical guidance from NTK theory

- **Learning rate warmup:** the NTK is random at initialisation; warming up η lets the kernel stabilise before large steps are taken.
- **Layer-wise learning rates:** different layers have very different NTK eigenvalue scales — separate η per layer (or LARS/LAMB) is justified by NTK analysis.
- **Spectral monitoring:** tracking $\|J\|_F$ (Frobenius norm of Jacobian) during training detects NTK drift in finite-width networks.

NTK-Informed Optimisation: Hierarchy of Approximations



Post-2018 Landscape: Overview

The NTK paper triggered an explosion of follow-up work. Major threads:

Thread	Key question
Lazy vs. feature learning	When does the NTK picture break down, and what replaces it?
μ P & Tensor Programs	Which parametrisations allow feature learning at infinite width?
Finite-width corrections	How do the $1/n$ corrections to the NTK affect generalisation?
Architecture extensions	NTK for CNNs, RNNs, Transformers, GNNs.
Practical limits	How well do NTK predictions match real networks on benchmarks?
Novel phenomena	Catapult phase, grokking, double descent through the NTK lens.

Lazy Training vs. Feature Learning

The “lazy” (NTK) regime (Chizat et al., 2019):

- Network stays close to its initialisation; parameters move $\mathcal{O}(1/\sqrt{n})$.
- NTK is approximately fixed $\Rightarrow$ training $\approx$ kernel regression.
- Triggered by: large initialisation scale, small learning rate, output rescaling.
- Network **cannot learn new features** — the kernel is data-independent.

The “rich” (feature learning) regime:

- Parameters move $\mathcal{O}(1)$; NTK evolves substantially.
- Network **adapts its representations** to the data.
- Empirically outperforms the NTK predictor on most real tasks.

Key result (Yang & Hu, 2021)

The lazy and rich regimes are **mutually exclusive** (dynamical dichotomy theorem): in the $n \rightarrow \infty$ limit, a network is either in the NTK regime (frozen kernel, no feature learning) or in the feature learning regime (μP), *never* both.

Practical implication

Real finite networks trained at standard scale usually *escape* the NTK regime early in training — NTK theory captures initialisation and very early training faithfully, but less so the full trajectory.

μ P Parametrisation and Tensor Programs

Yang & Hu (2020, 2021) introduced the **Maximal Update Parametrisation (μ P)** and the **Tensor Programs** framework to systematically classify all infinite-width limits.

Key results:

- Standard parametrisation and NTK parametrisation both land in the **lazy regime** at infinite width.
- μ P rescales each layer's weights by $1/n_\ell$ (not $1/\sqrt{n_\ell}$), enabling **maximal feature learning across all layers**.
- μ P is the *unique* parametrisation achieving non-trivial feature learning at $n \rightarrow \infty$.
- Depth-dependent scaling extensions (Yang et al., 2023) cover residual and transformer architectures.

Param.	Feature learning	NTK fixed?
Standard (NTK)	No	Yes
Mean field	Partial	Varies
μ P	Yes	No

Practical payoff

μ P enables **hyperparameter transfer**: tune on a small model, transfer to large. Used in Microsoft's μ Transfer (Yang et al., 2022).

Finite-Width Corrections to the NTK

The original NTK theory is exact only at $n = \infty$. Two key finite-width results:

Mean and variance of the NTK (Hanin & Nica, 2019):

- For ReLU networks: the standard deviation of the NTK scales as $\text{Std}[\Theta^{(L)}] \sim e^{L/n}$, i.e. **exponential in the depth-to-width ratio L/n** .
- Wide-but-not-deep networks: corrections small.
- Deep-but-narrow networks: corrections can dominate.

Feynman diagram approach (2024):

- Systematic $1/n$ perturbation theory for NTK statistics using Feynman-like diagrams.
- First-order corrections visibly improve alignment with finite-width samples.

Depth-width interplay

For scale-invariant activations (ReLU), $\Theta(x, x)$ has *no* finite-width corrections — but off-diagonal elements do. Critical initialisation that stabilises the infinite-width NTK also stabilises finite-width gradients, informing initialisation schemes.

Practical takeaway

Finite-width networks with $L/n \ll 1$ behave close to NTK theory. Most practical large models have $L/n \sim 0.01$ – 0.1 , so corrections are moderate but non-negligible.

NTK for Modern Architectures

Yang's **Tensor Programs II** (2020) extended the NTK to *any* architecture:

Architectures covered:

- Convolutional networks (CNN-NTK): competitive kernel for image tasks.
- Recurrent networks (RNN-NTK).
- **Transformer NTK**: infinite-width limit of attention + MLP blocks.
 - Linearised attention *does not* converge to its NTK limit at practical widths (attention cubes the Gram matrix condition number).
 - Standard softmax attention requires careful depth-and-width joint scaling.
- Graph Neural Network NTK (GNN-NTK): used to study oversmoothing and training dynamics on large graphs.
- Self-supervised learning NTK: infinite-width Barlow Twins converge to a fixed kernel (Eliezer et al., 2025).

Shaped Transformer (2023)

The “Shaped Transformer” (Noci et al., 2023) shows the standard NTK limit (fixed depth) fails to capture rank-collapse in deep attention. Joint depth-and-width limits are needed for faithful transformer theory.

Reference implementation

Open-source NTK computations for all these architectures are available via the `neural-tangents` JAX library (Novak et al., 2020), enabling practical use of NTK predictions.

Practical Performance Gap and Limitations

What NTK kernels do well:

- Small datasets: NTK often matches or beats finite-width networks (Arora et al., 2019).
- Theoretical upper bounds on generalisation error.
- Architecture search: ranking architectures by NTK spectrum correlates with test accuracy.
- NTK-CKA (centered kernel alignment) as a diagnostic for whether a network is in the lazy regime.

Where NTK predictions fail:

- **Large datasets:** finite-width networks outperform NTK kernel regression significantly (performance gap grows with dataset size).
- **Scale** and **fine-tuning:** pre-trained LLMs operate deep in the feature-learning regime.

The core limitation

The NTK describes a network that *never learns new features* — it is stuck with the random feature map at initialisation. Real networks succeed precisely by *changing* their feature map during training. The gap between NTK predictions and real networks *quantifies* how much feature learning matters for a given task.

Weighted NTK (WNTK): extends NTK to adjusted gradient descent (e.g. Adam), showing that the effective kernel under adaptive optimisers is a *reweighted* NTK (Zhang et al., 2023).

Novel Phenomena: Catapult Phase, Grokking, Double Descent

Three phenomena discovered after 2018 that the NTK framework partially explains — and partially fails to explain:

Catapult phase (Lewkowycz et al., 2020)

- Using learning rates *above* the NTK stability threshold ($\eta > 2/\lambda_{\max}$) causes the NTK to first “inflate” then settle at a **lower, flatter kernel**.
- Results in **better generalisation** than pure lazy training.
- Provides a mechanism for how large learning rates aid implicit regularisation.

Grokking (Power et al., 2022)

- Networks first memorise (NTK regime: high training accuracy, no generalisation), then *abruptly* generalise much later.
- Interpreted as a **delayed transition** from the lazy (NTK) regime to the feature-learning regime (Noether, 2024).
- Weight decay accelerates the transition by pushing the network out of the lazy regime.

Double descent (Belkin et al.; Nakkiran et al., 2019)

- Test error *decreases again* after the interpolation threshold.
- Analysed via NTK: in the overparametrised regime, the **minimum-norm interpolant** (= kernel regression with $\Theta_{\infty}^{(L)}$) generalises.
- The second descent coincides with NTK Gram matrix becoming well-conditioned.

NTK Theory: Status in 2025

What is now well-established:

- NTK provides a **rigorous, computable baseline** for understanding training dynamics and generalisation in the lazy regime.
- Finite-width corrections are under perturbative control for standard MLPs.
- NTK for arbitrary architectures (Tensor Programs): theoretical toolkit is mature.
- μP gives a principled path beyond the NTK to feature learning.
- NTK spectrum = direct predictor of training speed and early stopping.

What remains open:

- A full theory of the **feature-learning regime** at finite width.
- NTK for **transformers at joint depth-width scaling**.
- Quantitative connection between NTK structure and **in-context learning** in LLMs.
- The role of the NTK in **fine-tuning** and transfer learning.
- NTK description of **grokking timescales** and phase transition sharpness.

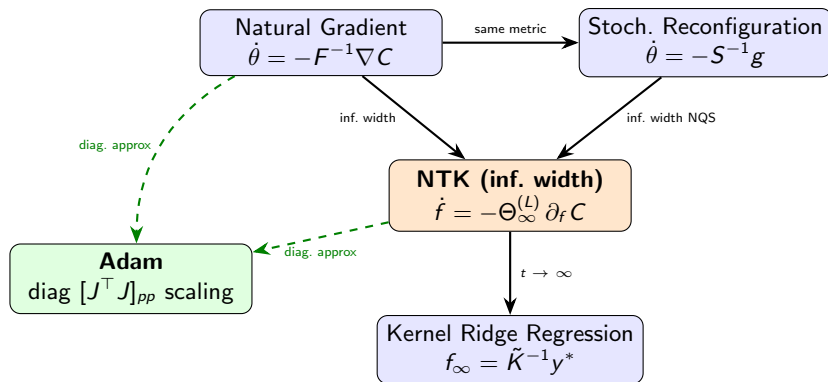
Bottom line

The NTK is the **linear theory** of deep learning — essential, exact in its regime, and a precise measure of its own limitations.

Summary of Main Results

- ➊ **NTK at initialisation** (Theorem 1): In the infinite-width limit, the NTK converges in probability to a fixed deterministic kernel $\Theta_{\infty}^{(L)}$, depending only on L , σ , and initialisation variance.
- ➋ **NTK during training** (Theorem 2): The NTK remains constant throughout training in the infinite-width limit — ANN training becomes **kernel gradient descent**.
- ➌ **Least-squares regression**: The network function satisfies a **linear ODE**; convergence follows the **kernel principal components** of the data.
- ➍ **Convergence guarantee**: Positive-definiteness of $\Theta_{\infty}^{(L)} \Rightarrow$ convergence to a global minimum.
- ➎ **Generalisation**: The limit f_{∞} equals kernel ridge regression / MAP estimation under a GP prior.
- ➏ **Early stopping**: Justified as avoiding fitting low-eigenvalue (noisy/high-frequency) directions.

Unifying Picture: Geometry of Learning



All methods share a common foundation: **preconditioned gradient descent on a Riemannian model manifold**. Adam provides the cheapest gradient-only approximation to the full NTK preconditioner.

Open Questions & Outlook

Theoretical extensions:

- Beyond infinite width: *finite-width corrections* (feature learning vs. lazy training).
- Mean-field / μ P parametrisation (Yang & Hu, 2021): NTK freezes $\Leftrightarrow$ no feature learning.
- NTK for CNNs, attention, transformers.
- Sharp sample complexity / generalisation bounds via NTK.

NQS / SR & optimiser connections:

- Infinite-width NQS: does S converge to a fixed kernel?
- Can NTK theory guide architecture choices for VMC?
- Spectral analysis of S for diagnosis of training pathologies.
- Connections to quantum geometry (quantum Fisher, Berry curvature).
- NTK-informed adaptive learning rates for SR (diagonal S shift vs. full inversion).
- μ P for NQS: is there a “maximal update” parametrisation for quantum variational methods?

Bottom line

The NTK provides a **mathematically rigorous bridge** between deep learning, kernel methods.

Key References

- L. Chizat, E. Oyallon & F. Bach. *On Lazy Training in Differentiable Programming*. NeurIPS 2019.
- G. Yang & E.J. Hu. *Feature Learning in Infinite-Width Neural Networks (μP)*. ICML 2021.
- B. Hanin & M. Nica. *Finite Depth and Width Corrections to the NTK*. ICLR 2020.
- G. Yang. *Tensor Programs II: NTK for Any Architecture*. NeurIPS 2020.
- A. Lewkowycz et al. *The Large Learning Rate Phase of Deep Learning: the Catapult Mechanism*. 2020.
- A. Power et al. *Grokking: Generalization Beyond Overfitting*. 2022.
- R. Novak et al. *Neural Tangents: Fast and Easy Infinite Neural Networks*. ICLR 2020.
- D.P. Kingma & J. Ba. *Adam: A Method for Stochastic Optimization*. ICLR 2015.
- J. Martens & R. Grosse. *Optimizing Neural Networks with K-FAC*. ICML 2015.
- A. Jacot, F. Gabriel, C. Hongler. *Neural Tangent Kernel: Convergence and Generalization in Neural Networks*. NeurIPS 2018.
- R.M. Neal. *Bayesian Learning for Neural Networks*. Springer, 1996.
- J.H. Lee et al. *Deep Neural Networks as Gaussian Processes*. ICLR 2018.
- S. Amari. *Natural Gradient Works Efficiently in Learning*. Neural Computation, 1998.
- S. Sorella. *Green Function Monte Carlo with Stochastic Reconfiguration*. PRL, 1998.
- G. Carleo & M. Troyer. *Solving the quantum many-body problem with artificial neural networks*. Science, 2017.
- G. Yang & E.J. Hu. *Tensor Programs IV: Feature Learning in Infinite-Width Neural Networks*. ICML 2021.
- F. Becca & S. Sorella. *Quantum Monte Carlo Approaches for Correlated Systems*. Cambridge, 2017.

Thank you

Questions?